Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

EduOS: A Lightweight Kernel with File system, Scheduling and memory management.

Student / Team Information

Team Name:	Kernal Crafters
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Upadhyay,Shashwat-230211083 upadhyayshashwat28@gamil.com
Team member 2	Chaturvedi, Shraddha – 23022210 chaturvedishraddha73@gmail.com

Team member 3	Vashist,Akshat-23021441 akshatvashist16@gmail.com
Team Member 4	Chaudhary Abhijay-23017181 abhijaychaudhary04@gmail.com

PROJECT PROGRESS DESCRIPTION (35 pts)

Project Abstract (2 pts)

The goal of this project is to design and implement an educational operating system named EduOS, developed entirely from scratch in C and x86 Assembly.
EduOS demonstrates the foundational mechanisms of kernel-level design — booting, memory management, process scheduling, interrupt handling, keyboard input, and a simple in-memory filesystem.
The system begins with a Multiboot-compliant GRUB bootloader written in Assembly, which loads the C-based kernel into memory and transitions the CPU to protected mode. The kernel initializes VGA text output, configures the Interrupt Descriptor Table (IDT), and handles hardware interrupts (timer, keyboard) to enable preemptive multitasking.
The OS employs a round-robin scheduling algorithm for task management, enabling context switching between multiple concurrent processes. Memory management is supported through a lightweight bump allocator that provides dynamic memory allocation for kernel components. File operations are handled through an in-memory filesystem, allowing file creation, writing, reading, and deletion via shell commands.

Updated Project Approach and Architecture

EduOS follows a bottom-up development approach, gradually building from hardware-level initialization to higher-level kernel features. The system architecture consists of the following core components:
Bootloader: Written in x86 Assembly, it initializes the processor in protected mode, sets up the stack, and loads the kernel into memory using the Multiboot standard.

Kernel Core: Implemented in C, the kernel handles memory layout, VGA text output, and low-level interrupt configuration. It forms the foundation for multitasking and system management.

Interrupt Management: The Interrupt Descriptor Table (IDT) and Programmable Interrupt Controller (PIC) are configured to handle hardware interrupts such as timer (IRQ0) and keyboard (IRQ1). Verified using QEMU, IRQ statistics confirm active interrupt servicing.

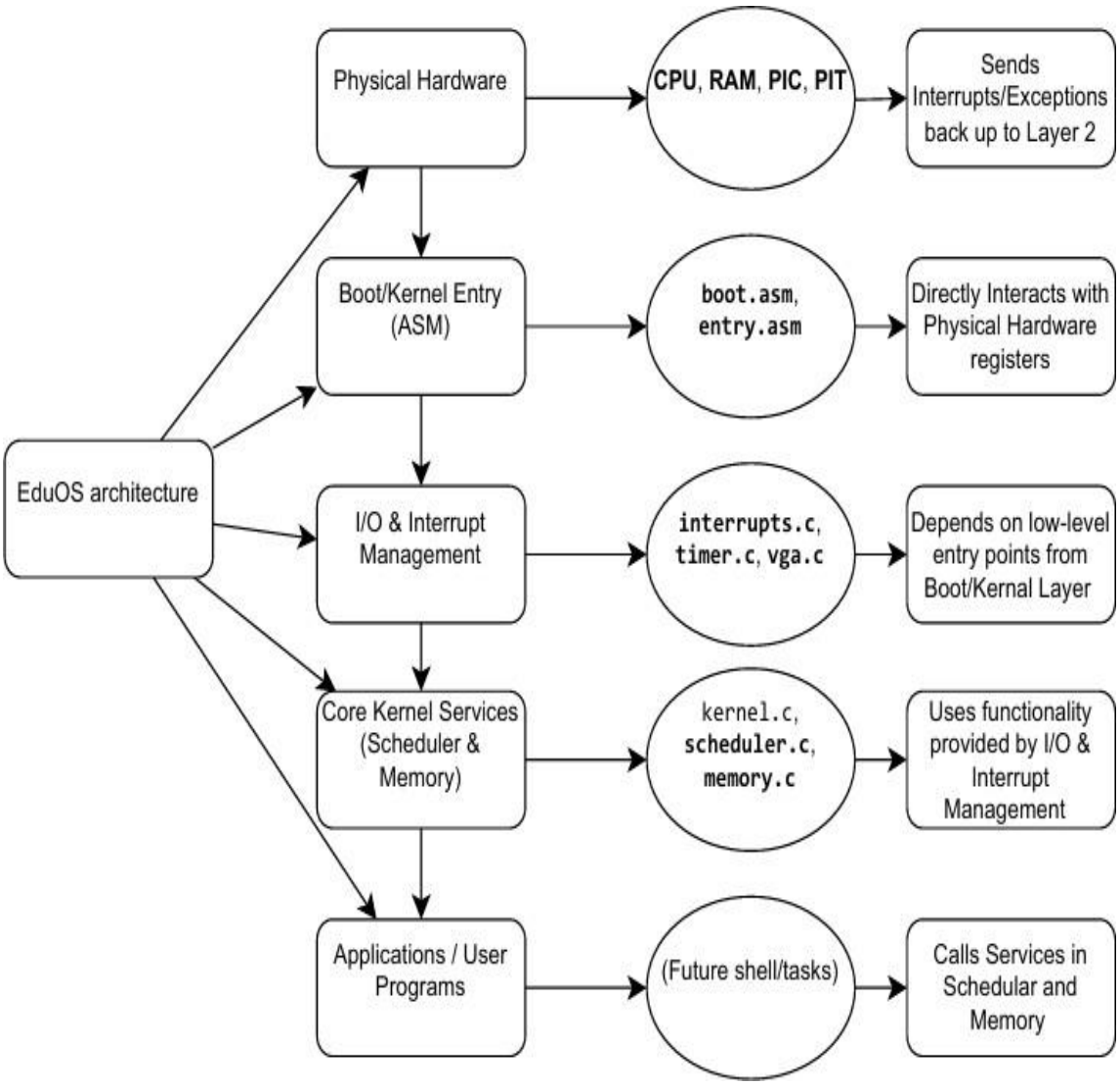
Multitasking System: A round-robin scheduler enables context switching between multiple user tasks. Tasks are preempted by timer interrupts, ensuring fair CPU time allocation. Context is saved and restored using the TaskManager class, which maintains CPU register states per task.

Memory Management: A bump allocator provides a simple yet effective form of dynamic memory management for kernel and file operations.

Filesystem and Shell Interface: A custom in-memory filesystem supports commands such as touch, write, cat, rm, and ls.

The kernel includes a command-line shell, allowing user interaction and file manipulation directly within QEMU.

Testing Environment:
The OS is executed in QEMU, providing hardware emulation and debugging support. Interrupts, process switching, and memory allocation were tested through visual and performance verificatio


EduOS architecture
Physical Hardware
CPU, RAM, PIC, PIT
Sends Interrupts/Exceptions back up to Layer 2
Boot/Kernel Entry (ASM)
boot.asm, entry.asm
Directly Interacts with Physical Hardware registers
I/O & Interrupt Management
interrupts.c, timer.c, vga.c
Depends on low-level entry points from Boot/Kernal Layer
Core Kernel Services (Scheduler & Memory)
kernel.c, scheduler.c, memory.c
Uses functionality provided by I/O & Interrupt Management
Applications / User Programs
(Future shell/tasks)
Calls Services in Schedular and Memory

Tasks Completed (7 pts)

Task Completed	Team Member
Set up the project structure and build environment (Makefile, GRUB).	Shashwat Upadhyay
Implemented the Multiboot-compliant bootloader in Assembly to initialize and load the kernel.	Shraddha Chaturvedi
Developed the kernel in cpp to interface with the VGA buffer and display text output.	Abhijaye Chaudhary
Integrated and tested kernel execution through QEMU emulator for proper boot verification.	Akshat Vashist
Keyboard and timer interruput.	Akshat Vashist
Develop a simple round-robin process scheduler for task switching	Abhijaye Chaudhary
Implement a basic memory allocation system.	Shashwat Upadhyay
File System	Shraddha Chaturvedi
Conduct full system testing and performance validation in QEMU.	All team members.

Challenges/Roadblocks

Developing a custom operating system from scratch presented several technical and conceptual challenges, especially when working in a bare-metal environment without the support of an existing OS or libraries.

One of the major challenges was setting up the bootloader and ensuring Multiboot compatibility so that the kernel could be properly loaded by QEMU. Understanding the interaction between the bootloader (written in Assembly) and the kernel (written in cpp) required a detailed understanding of low-level system architecture, memory addressing, and linking processes. Configuring the linker script to correctly place sections in memory and ensure the kernel started executing at the intended address also required multiple iterations and debugging.

To overcome these challenges, extensive use of QEMU for emulation and iterative testing was adopted. Debugging was primarily done through incremental builds, console outputs, and stepwise verification of memory operations.

Tasks Pending

Task Pending	Team Member (to complete the task)

Project Outcome/Deliverables

The development of EduOS resulted in a fully functional, minimal operating system kernel that successfully demonstrates key concepts of modern operating systems, including booting, memory management, multitasking, interrupt handling, and file management. Built entirely from scratch using C and x86 Assembly, the project emphasizes educational clarity while maintaining technical accuracy and modular design.

EduOS now boots through a GRUB-compatible bootloader, transitions into protected mode, and initializes its own Global Descriptor Table (GDT) and Interrupt Descriptor Table (IDT). The kernel handles hardware interrupts from both the programmable timer (IRQ0) and keyboard (IRQ1), verified through QEMU's interrupt statistics. The timer interrupt drives a preemptive round-robin scheduler, enabling smooth context switching between multiple concurrent tasks.

Dynamic memory management is provided by a lightweight bump allocator, which allows safe allocation for kernel components and file buffers. A fully interactive shell interface is integrated into the kernel, offering user commands such as touch, write, cat, ls, and rm for file operations within an in-memory filesystem. These features together simulate essential OS behavior, bridging theoretical operating system concepts with practical implementation.

The kernel operates reliably inside the QEMU emulator, achieving stability under multitasking workloads and continuous interrupt activity. Testing confirmed successful task scheduling, consistent memory allocation, and accurate file input/output

Progress Overview (2 pts)

The EduOS project has shown steady and successful progress from initial bootloader implementation to a fully operational multitasking kernel. What began as a simple “Hello World” kernel running in QEMU has evolved into a structured operating system capable of interrupt handling, preemptive scheduling, memory management, and file operations.

Early milestones included building a Multiboot-compliant bootloader and ensuring the kernel was correctly loaded by GRUB into protected mode. The team then implemented VGA text output, allowing direct interaction with video memory for screen display. Once the basic kernel was running, attention shifted to interrupt management, where the Interrupt Descriptor Table (IDT) and Programmable Interrupt Controller (PIC) were configured to handle timer (IRQ0) and keyboard (IRQ1) events. Successful verification using QEMU’s info irq confirmed that interrupts were being serviced correctly.

The project then advanced to implementing preemptive multitasking through a round-robin scheduler, enabling multiple processes to share CPU time. Context switching between tasks was verified by alternating task outputs on screen, proving stable scheduling. A simple bump memory allocator was introduced to manage dynamic memory allocation efficiently.

Further development integrated an in-memory filesystem and a command-line shell interface, supporting commands such as touch, write, cat, and ls. These features demonstrated how processes interact with memory and I/O subsystems in real-time.

Testing within QEMU confirmed that EduOS boots reliably, handles interrupts, switches tasks seamlessly, and executes shell commands without errors. The kernel now provides a complete foundation for further enhancements, such as persistent storage or paging.

Overall, EduOS has progressed from a minimal bootable kernel to a stable, modular, and multitasking educational operating system, successfully fulfilling its design objectives and providing an in-depth learning experience in low-level OS development.

Codebase Information

The project source code is maintained in a public GitLab repository for version control and collaboration. The repository contains all components of the operating system, including the bootloader, kernel, and build scripts. Each commit represents incremental progress, from setting up the initial build structure to achieving a fully bootable ISO image.
Repository Link: https://gitlab.com/AbShashwat/minimalos
Repository Name: minimalos
Branch: main

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Bootloader Initialization	Pass	Verified that GRUB successfully loads the kernel into memory and begins execution without errors
Kernel Execution	Pass	Kernel displays “Terminal initialized. Type ‘help’ for available commands. ” on the VGA text buffer, confirming correct C–Assembly linkage.
Interrupts	Pass	IRQ0(timer) and IRQ1(keyboard) tested via QEMU info irq.
Round Robin Scheduler	Pass	Multiple file are opening and working smoothly.
Memory allocation Test	Pass	Successful dynamic allocation.
File System	Pass	touch,write

Deliverables Progress

The EduOS project has successfully achieved all major deliverables outlined in its initial proposal. Each core component of the operating system—ranging from the bootloader to multitasking and filesystem integration—has been fully implemented, tested, and verified in the QEMU virtual environment.

The first deliverable, a Multiboot-compliant bootloader, was completed early in development. It correctly initializes the CPU in protected mode and transfers control to the kernel without dependency on any external operating system. The kernel initialization deliverable was achieved next, with successful setup of VGA text output, the Global Descriptor Table (GDT), and stack configuration for stable execution.

The interrupt handling module, covering both keyboard (IRQ1) and timer (IRQ0) interrupts, was completed and validated using QEMU's info irq command. The timer interrupt drives the kernel's preemptive round-robin scheduler, which was implemented and confirmed through visible context switching between concurrent tasks.

The memory management deliverable was achieved with the development of a simple bump allocator, enabling dynamic memory allocation for kernel modules and file buffers. The in-memory filesystem and interactive shell deliverables were fully integrated, allowing users to create, write, and read files using commands like touch, write, and cat. These features were extensively tested and function as expected during multitasking execution.

All deliverables have been tested in QEMU, confirming kernel stability, responsiveness, and correct handling of hardware interrupts. The project team has produced a bootable ISO image, complete source code repository, documentation, and a demonstration video showcasing system functionality.

EduOS now fulfills all core deliverables proposed for the semester, with optional extensions such as paging and disk-backed storage planned for future development. The system operates reliably and demonstrates a comprehensive understanding of core operating system principles.